

Ejercicio de Programación en Python

En esta asignación, se proporcionarán cinco programas en Python como ejemplo, y se pedirá a los estudiantes que trabajen en grupos de tres para crear otros cinco programas similares. Todos los programas deben involucrar el uso de funciones y al menos uno de ellos debe utilizar listas.

Ejemplos de Programas en Python usando funciones

1. Escribe una función que reciba una lista de números y devuelva la suma de los números pares en la lista, pero que implemente un parámetro por defecto, que al cambiarlo sume también los impares

```
def sumar_pares(lista, incluir_impares=False):
    suma = 0
    for numero in lista:
        if numero % 2 == 0:
            suma += numero
        elif incluir_impares:
            suma += numero
    return suma

numeros = [1, 2, 3, 4, 5]
resultado1 = sumar_pares(numeros)
resultado2 = sumar_pares(numeros, incluir_impares=True)
print(resultado1)    # Imprime: 6
print(resultado2)    # Imprime: 9
```

2. Escribe una función que reciba una lista de cadenas y devuelva una nueva lista con las cadenas que tengan una longitud mayor o igual a 5. Utiliza los temas de funciones con listas y condicionales.

```
def filtrar_cadenas_largas(lista):
    cadenas_largas = []
    for cadena in lista:
        if len(cadena) >= 5:
            cadenas_largas.append(cadena)
    return cadenas_largas

palabras = ["hola", "mundo", "bienvenidos", "python"]
```

```
resultado = filtrar_cadenas_largas(palabras)
print(resultado) # Imprime: ['mundo', 'bienvenidos', 'python']
```

3. Escribe una función que reciba una lista de números y devuelva el promedio. Utiliza los temas de funciones con listas y operadores matemáticos.

```
def calcular_media(lista):
    suma = sum(lista)
    media = suma / len(lista)
    return media

numeros = [1, 2, 3, 4, 5]
resultado = calcular_media(numeros)
print(resultado) # Imprime: 3.0
```

4. Escribe una función que reciba una lista de números y devuelva la suma de los elementos de la lista que están en posiciones pares. Utiliza los temas de funciones con listas y ciclos.

```
def sumar_pares_en_posicion(lista):
    suma = 0
    for i in range(len(lista)):
        if i % 2 == 0:
            suma += lista[i]
    return suma

numeros = [1, 2, 3, 4, 5, 6, 7]
resultado = sumar_pares_en_posicion(numeros)
print(resultado) # Imprime: 9
```

5. Escribe una función que reciba una cadena y devuelva otra cadena con todas las vocales en mayúsculas. Utiliza los temas de funciones con cadenas y ciclos

```
def convertir_vocales_a_mayusculas(cadena):
    nueva_cadena = ""
    for caracter in cadena:
        if caracter in "aeiouAEIOU":
            nueva_cadena += caracter.upper()
        else:
            nueva_cadena += caracter
    return nueva_cadena

texto = "Hola mundo"
resultado = convertir_vocales_a_mayusculas(texto)
print(resultado) # Imprime: HOIA mUndO
```

Ejercicios Propuestos(puede eliminar 1 y hacer 4, total de 10 puntos)

1. Crear una función que encuentra los números primos en un rango de números especificado por el usuario.
2. Crear una función que reciba como parámetro una lista de strings y devuelva una nueva lista con los elementos ordenados alfabéticamente.
3. Crear una función que reciba como parámetro una lista de strings y devuelva una nueva lista con las palabras que contengan una letra especificada.
4. Crear una función que reciba como parámetro una lista de números y devuelva el número más grande de la lista.
5. Crear una función para ordenar una lista de números de menor a mayor.

Ejemplos Funciones Recursivas en python:

Ejemplo1: Factorial recursivo

Escribe una función recursiva en Python que calcule el factorial de un número entero. El número debe ser ingresado por el usuario y el resultado debe ser mostrado en pantalla.

```
def factorial(n):  
    """  
    Esta función calcula el factorial de un número de forma recursiva.  
    """  
    # Caso base  
    if n == 1:  
        return 1  
    # Caso recursivo  
    else:  
        return n * factorial(n-1)  
  
# Ejemplo de uso  
resultado = factorial(5)  
print(resultado) # Debería imprimir 120
```

Este programa calcula el factorial de un número utilizando una función recursiva. En este programa, la función `factorial()` es recursiva. El caso base es cuando `n` es

igual a 1, en ese caso, la función devuelve 1. En caso contrario, la función se llama a sí misma con el parámetro $n-1$ y multiplica el resultado por n .

Ejemplo2: Fibonacci recursivo

Escribe una función recursiva en Python que genere la secuencia de Fibonacci hasta un número entero ingresado por el usuario. El número debe ser ingresado por el usuario y la secuencia debe ser mostrada en pantalla.

```
def fibonacci(n):  
    """  
    Esta función genera la secuencia de Fibonacci de forma recursiva.  
    """  
    # Casos base  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    # Caso recursivo  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)  
  
# Ejemplo de uso  
resultado = fibonacci(10)  
print(resultado) # Debería imprimir 55
```

Este programa genera la secuencia de Fibonacci utilizando una función recursiva.

En este programa, la función `fibonacci()` es recursiva. Los casos base son cuando n es igual a 0 o 1, en esos casos, la función devuelve 0 o 1, respectivamente. En caso contrario, la función se llama a sí misma con los parámetros $n-1$ y $n-2$ y devuelve la suma de los dos resultados.

Ejemplo3: Búsqueda binaria recursiva

Escribe una función recursiva en Python que implemente la búsqueda binaria en una lista de números enteros. La función debe recibir como parámetros la lista de números y el número a buscar, y debe devolver el índice del número en la lista o -1 si el número no se encuentra en la lista. La lista de números debe ser ingresada por el usuario y el número a buscar debe ser generado aleatoriamente. El resultado de la búsqueda debe ser mostrado en pantalla.

```
def busqueda_binaria(lista, valor, inicio=0, fin=None):
    """
    Esta función busca un valor en una lista utilizando la búsqueda binaria de
    forma recursiva.
    """
    # Si no se especifica el valor de fin, se toma el valor máximo
    if fin == None:
        fin = len(lista) - 1

    # Caso base: el valor no está en la lista
    if inicio > fin:
        return -1

    # Calculamos el índice medio
    medio = (inicio + fin) // 2

    # Caso base: el valor está en el índice medio
    if lista[medio] == valor:
        return medio

    # Caso recursivo: buscamos en la mitad izquierda o derecha de la lista
    elif lista[medio] > valor:
        return busqueda_binaria(lista, valor, inicio, medio-1)
    else:
        return busqueda_binaria(lista, valor, medio+1, fin)

# Ejemplo de uso
lista = [1, 3, 5, 7, 9]
indice = busqueda_binaria(lista, 7)
print(indice) # Debería imprimir 3
```

Este programa busca un elemento en una lista utilizando una función recursiva que implementa la búsqueda binaria.

En este programa, la función `busqueda_binaria()` es recursiva. Los casos base son cuando el valor no está en la lista (el índice de inicio es mayor que el índice de fin) o cuando

Ejercicios (puede eliminar 1 y hacer sólo 1, el puntaje del ejercicios es 10)

Ejercicio 1: Suma recursiva

Escribe una función recursiva en Python que calcule la suma de los primeros n números enteros. El valor de n debe ser ingresado por el usuario y el resultado debe

ser mostrado en pantalla. Utiliza la recursividad para calcular la suma.

Ejercicio 2: Potencia recursiva

Escribe una función recursiva en Python que calcule la potencia de un número entero. El número base y el exponente deben ser ingresados por el usuario y el resultado debe ser mostrado en pantalla. Utiliza la recursividad para calcular la potencia.

Criterios de Evaluación

Los grupos serán evaluados en función de la calidad de sus programas y su capacidad para cumplir con los siguientes criterios:

1. **Correctitud de la solución propuesta (40%)** : se evaluará si los programas funcionan correctamente y producen los resultados esperados.
2. **Uso efectivo de las funciones y las listas (30%)** : se evaluará la calidad y eficacia del uso de las funciones y listas en los programas.
3. **Legibilidad y estructura del código (30%)** : se evaluará la legibilidad y organización del código, incluyendo la documentación, la selección de nombres de variables y la estructura general del programa.

Los criterios de evaluación se ponderarán de la siguiente manera: Correctitud de la solución propuesta (40%), Uso efectivo de las funciones y las listas (30%) y Legibilidad y estructura del código (30%).